Methodology

The loop is implemented through the project source surface summarized by the validated run artifacts. The project scripts remain thin dispatchers; reusable behavior writes `output/data/autoresearch_loop.json`, `output/data/ml_task_results.json`, `output/figures/figure_registry.json`, `output/data/autoresearch_phase_ledger.json`, and `output/data/manuscript_variable_provenance.json`.

Task And Data I

The task is small MNIST neural-network classification. The default configuration loads data/mnist_small.npz, with provenance in data/mnist_small_provenance.json, and uses seed 20260525. The subset contains 2000 training images and 500 test images, with 10 classes present in both splits and image shape 28 by 28. The provenance artifact records upstream source-file hashes, the subset seed, class counts, and the compressed subset hash. The default pipeline never downloads data at runtime. The train and test splits contain 200 and 50 examples per class, respectively. The registered class-balance diagnostic is fig. 1, and the dataset contact sheet is fig. 2.

Task And Data I

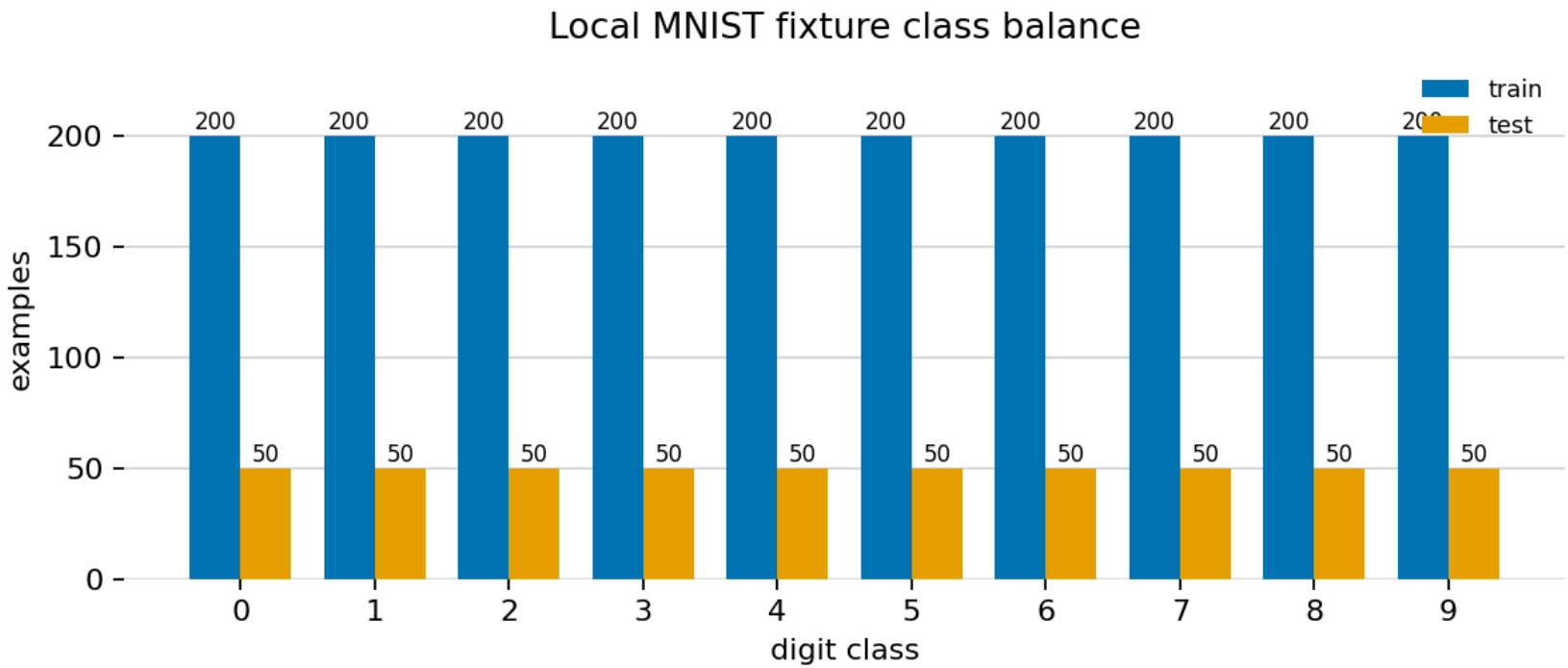


Figure 1: Train and test class counts from output/data/ml_class_balance.json; the local fixture contains 2000 train and 500 test examples across 10 classes. Generation method: Grouped train/test class-count bars from the local MNIST fixture. Registry metadata records the generation method, source artifact, and claim boundary for validation.

Task And Data II

Table 1: Local fixture class-balance table from `output/data/ml_class_balance.json`; counts describe the offline fixture used by this run.

Split	Class	Count	Fraction
train	0	200	10.0%
train	1	200	10.0%
train	2	200	10.0%
train	3	200	10.0%
train	4	200	10.0%
train	5	200	10.0%
train	6	200	10.0%
train	7	200	10.0%
train	8	200	10.0%
train	9	200	10.0%
test	0	50	10.0%
test	1	50	10.0%
test	2	50	10.0%
test	3	50	10.0%
test	4	50	10.0%
test	5	50	10.0%
test	6	50	10.0%
test	7	50	10.0%
test	8	50	10.0%
test	9	50	10.0%

Task And Data III

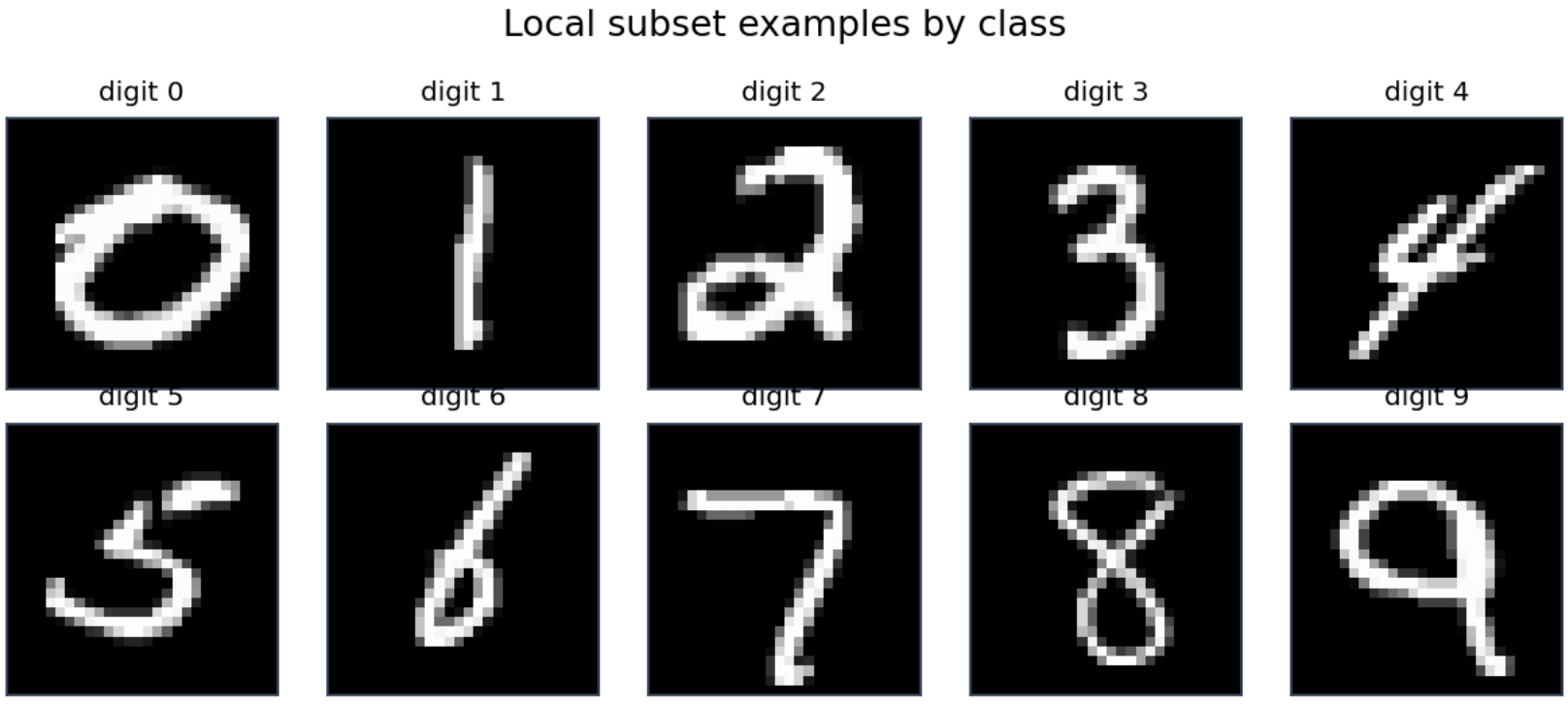


Figure 2: Deterministic class-balanced contact sheet for MNIST handwritten digit database from data/mnist_small.npz and data/mnist_small_provenance.json; the figure documents the local subset used by the offline run. Generation method: Class-balanced contact sheet from fixed local MNIST arrays. Registry metadata records the generation method, source artifact, and claim boundary for validation.

The baseline is `nearest_centroid_baseline` (`nearest-centroid`). The bounded candidate set is configured by the task artifact and covers `MLP`, `nearest-centroid`, `softmax regression`, `tiny patch-attention`, including `Tiny patch-attention classifier` and at least one deferred proposal. The transformer-style candidate borrows the patch-token representation for comparison while staying inside the configured iteration budget. This is intentionally a small configured model, not a claim about full-scale image-transformer performance.

Bounded Loop I

The run follows 7 configured stages:

Bounded Loop I

- Resolve the human-authored program, project topic, and research questions.
- Build an `AutoResearchPlan` from the domain profile, experiment plan, and pipeline DAG.
- Validate exact stage-gate names declared in `autoresearch.yaml`: an entry Validation rejects unknown stage names — misspelling a gate name halts configuration before any phase executes. naming a stage absent from `pipeline.yaml` is a validation error (`AUTORESEARCH.STAGE_UNKNOWN`), never silently skipped. The claim substance gate has negative controls in `tests/test_gate_negative_controls.py`, which feeds missing, hollow, and garbage evidence artifacts and asserts each claim is rejected.
- Evaluate the configured `MNIST` candidate set up to the configured iteration budget.
- Generate claims only from configured questions and local artifact paths.
- Write data, reports, figures, benchmark scores, and review packets under the declared output contract.
- Run strict AutoResearch readiness validation and write readiness reports.

Bounded Loop I

Candidates are declared in the proposal ledger and resolved from the task configuration for execution. Each executable candidate declares a model type, seed, training schedule, and model-specific parameters. The configured training policy includes learning-rate decay `0.995` and gradient clipping norm `5`, both recorded from the validated task run rather than described as a free-form manuscript setting. The loop evaluates at most `4` configured iterations, selects the highest `test_accuracy`, and breaks ties by lower parameter count and identifier. Candidates outside the budget are recorded as deferred; the run-derived status summary is `accepted: 1, deferred: 1, rejected: 3`.

Bounded Loop I

The closure is concrete and file-backed. `output/data/research_program.json` constrains proposals; proposal records feed the bounded evaluation; evaluation writes `output/data/ml_candidate_ledger.json` and `output/data/run_ledger.json`; ledgers support artifact-linked claims; claims hydrate the manuscript through `output/data/manuscript_variables.json`; readiness validation writes `output/reports/autoresearch_readiness.json`; loop settlement is recorded in `output/data/autoresearch_phase_ledger.json`; and review state is captured in `output/data/review_decisions.json`. The loop therefore maintains an inspectable research process around a small metric result without making the process self-approving or opaque.

The concrete closure is intentionally close to research-object and workflow-run provenance practice. The artifact manifest lists generated objects and checksums; the figure registry binds captions to source artifacts; the variable provenance sidecar records the source pointer for each injected token or table; and the review packet keeps human decisions outside generated approval. This is a minimal local analogue of machine-readable provenance rather than a claim of full research-object packaging.

The generated schema manifest at `output/data/autoresearch_schema_manifest.json` records 31 schema-versioned governance payload(s) plus documented generic-table exemptions. The local research-object manifest at `output/data/research_object_manifest.json` packages observed project paths, hashes, the evidence registry, the source ledger, the schema manifest, and the manual approval state. It is deliberately named a local research-object manifest, not RO-Crate or SLSA compliance.

Safety Controls I

The default autonomy level is `proposal_only`. The run ledger records `0` LLM calls and USD `0.00` cost. The edit allowlist is restricted to the public project source and manuscript surfaces, plus the task configuration file, and the pipeline never executes generated code. Review gates are emitted with deferred decisions so that validation can confirm the gates exist without pretending that the machine approved publication.

Safety Controls I

Publication approval is a non-generated input. The run may read `human_review.yaml` and copy its state into review payloads, but generated readiness cannot set publication approval by itself; the default local review state remains `false`.

Disclosure: `AI-assisted AutoResearch` status is declared for this exemplar because it models machine-produced plans, ledgers, reports, and manuscript variables as review inputs rather than autonomous approval.

Adversarial And Supply-Chain Controls I

The security layer is configured as `local_deterministic` with network policy `default_offline`, integrity algorithm `sha256`, external signing `false`, and framework labels `STRIDE, MITRE_ATT&CK_T1195`. Its scope is `Local research-artifact integrity evidence for this deterministic public exemplar`. These values are generated from `output/data/autoresearch_security_profile.json`; the default run remains offline and deterministic.

Adversarial And Supply-Chain Controls I

The local threat model covers 7 asset(s), 7 threat row(s), and 7 control row(s). The security artifacts are written to `output/data/autoresearch_threat_model.json`, `output/data/autoresearch_supply_chain_inventory.json`, `output/data/autoresearch_integrity_attestation.json`, and `output/reports/autoresearch_security_review.md`. The control-matrix figure is fig. 3. The table and figure are generated from the threat model and inventory, not manually maintained.

Adversarial And Supply-Chain Controls I

Local security controls by evidence surface

Control	Evidence surface	Framework cue	Status
offline	network policy	NIST SP 800 207	implemented
budget	run ledger	NIST SP 800 218 SSDF	implemented
hashes	integrity attestation	SLSA SPEC	implemented
inventory	supply chain inventory	SLSA SPEC	implemented
review	review decisions	NIST SP 800 218 SSDF	implemented
allowlist	idea ledger	MITRE ATTACK T1195	implemented
boundary	security profile	NIST SP 800 207	implemented

local checksum and review controls only

Figure 3: Local security-control matrix from output/data/autoresearch_threat_model.json; controls map NIST, SLSA, and ATT&CK-inspired safeguards to deterministic evidence surfaces without claiming production security certification. Generation method: structured control matrix with separate control, evidence, framework, and status columns. Generation method: Structured control matrix from local threat-model controls. Registry metadata records the generation method, source artifact, and claim boundary for validation.

Adversarial And Supply-Chain Controls II

Table 2: Local security artifacts generated for the bounded AutoResearch run.

Security artifact	Path	Summary
profile	security profile	local_deterministic
threat model	threat model	default_offline
inventory	supply inventory	14 inputs
inventory export	inventory export	local non-SBOM export
attestation	integrity attestation	passed
review	security review	human review input

Adversarial And Supply-Chain Controls I

Table 3: Threat-model rows from `output/data/autoresearch_threat_model.json`; ATT&CK labels scope supply-chain compromise analogies.

Threat	STRIDE	ATT&CK	Scenario	Residual risk
dataset tamper	Tampering	T1195	A local fixture could be replaced or edited before analysis.	Residual risk remains if a reviewer ignores checksum drift.
config drift	Tampering	T1195.001	Task settings could silently change candidate scope, budgets, or diag...	Configuration review is still a human responsibility.
source edit	Elevation of privilege	T1195.001	Source changes could bypass the thin-script and no-generated-code bou...	The default run does not perform full static application security tes...
output tamper	Repudiation	T1195.002	Generated reports or figures could be edited after analysis but befor...	Local checksum evidence is not externally signed.
manuscript injection	Information disclosure	T1195.002	Manual prose could hard-code run facts that bypass validated variables.	Stable scholarly prose and citekeys remain manually authored.
self approval	Spoofing	T1195	Generated review packets could be mistaken for human publication appr...	Publication remains outside the automated loop.
build assumption	Denial of service	T1195.003	A local or CI build context could omit checks or run stale generated...	The exemplar does not sign build logs or isolate runners.

Positioning Against Autonomous Science Systems I

The implementation should be read as a bounded local analogue of current AutoResearch trends, not as a miniature autonomous scientist. Artifact manifests, evidence registries, review gates, and manuscript hydration provide machine-readable governance surfaces for one offline project run. They do not perform autonomous literature mining, proof search, live agent orchestration, runtime dataset expansion, self-modifying code search, or self-approval of publication claims.

The experiment writes output/data/ml_task_results.json, output/data/ml_candidate_ledger.json, output/data/ml_confusion_matrix.csv, output/data/ml_training_history.csv, output/data/ml_error_examples.json, output/data/ml_prediction_records.json, output/data/ml_classification_diagnostics.json, output/data/ml_candidate_intervals.json, output/data/ml_class_balance.json, output/data/ml_calibration_report.json, output/data/ml_calibration_bin_intervals.json, output/data/ml_robustness_report.json, output/data/ml_probability_diagnostics.json, output/data/ml_bootstrap_intervals.json, output/data/ml_paired_comparison.json, output/data/ml_candidate_rank_stability.json,

output/data/ml_statistical_summary.json, output/data/ml_training_diagnostics.json, output/reports/ml_benchmark_score.json, output/data/benchmark_boundary.json, output/data/figure_quality_report.json, and registered figures through output/figures/figure_registry.json. The diagnostic payloads preserve probabilities, confidence and margin summaries, class metrics, calibration bins, confusion-pair summaries, train/test gaps, deterministic no-retrain perturbation scores, bootstrap intervals, paired baseline comparison, rank-stability frequencies, calibration-bin Wilson intervals, selective accuracy thresholds, Brier score, negative log likelihood, top-2 accuracy, probability-quality comparisons, learning-rate traces, best-epoch markers, final learning rates, and train-test gap summaries without serializing model weights. The benchmark score combines metric improvement, budget compliance,

offline execution, transformer-candidate coverage, and candidate-selection status. The benchmark-boundary artifact records fixture scope, metric direction, candidate families, budget, statistical-method artifacts, and explicit non-claims, so benchmark-adjacent statistics remain local readiness diagnostics rather than broad empirical or publication claims. Manuscript variables are hydrated from these artifacts, and readiness validation checks `output/reports/evidence_registry.json`, `output/reports/artifact_manifest.json`, method ledgers, review gates, benchmark outputs, the phase ledger, figure-quality report, and AI-assisted disclosure before rendering is treated as ready for review. The reviewer-facing `output/data/autoresearch_evidence_overview.json` then summarizes readiness versus publication approval, claim-evidence rows, source-ledger status, benchmark boundary issues, and security/integrity

status without granting approval. The generated tables and figure blocks used in sec. 4 are sourced through `output/data/manuscript_variable_provenance.json` and `output/data/manuscript_figure_blocks.json`, so captions, artifact tables, and run-derived result statements share the same validated artifact base.

The figure registry also carries the method contract for each visualization: source artifact, generated file, rendering method, and claim boundary. The rendered method table below is generated from that registry rather than maintained manually.

Table 4: Registry-backed figure methods from `figure_registry.json`; full validation hooks, alt text, and claim boundaries remain in the registry.

Figure	Source	Method	Scope
fig. 14	candidate ledger	Candidate lifecycle status-count bar chart.	Lifecycle counts describe bounded orchestration, not autonomous approval.
fig. 26	loop	File-backed process-flow diagram from final loop state.	The workflow is file-backed and inspectable but not self-approving.
fig. 27	integrity attestation	Local checksum attestation chain with checked, missing, and mismatch counts.	Integrity checks are local SHA-256 observations and are not externally signed provenance.
fig. 3	threat model	Structured control matrix from local threat-model controls.	Controls are local research-artifact safeguards, not production security certification.
fig. 25	loop	Horizontal count summary from final loop metrics.	Readiness artifacts summarize local validation only; publication approval is human.
fig. 21	bootstrap intervals	Horizontal percentile-bootstrap interval plot.	Bootstrap intervals summarize local test-set resampling only.
fig. 15	calibration report	Reliability curve with confidence-bin support histogram.	Calibration values describe the fixed local split only.
fig. 7	candidate rank stability	Bootstrap top-rank frequency and mean-rank comparison.	Rank stability describes local resampling behavior, not model-selection certainty.
fig. 6	candidate intervals	Lollipop accuracy comparison with Wilson interval error bars and direct labels.	Scores apply only to the fixed local subset and configured candidates.

Evidence And Scoring II

fig. 16	class diagnostics	Per-class precision, recall, and F1 heatmap.	Class metrics diagnose this run only and are not full-dataset estimates.
fig. 12	task results	Log-parameter scatter plot against held-out accuracy.	The plot compares this bounded candidate set and does not infer a scaling law.
fig. 8	confusion matrix	Row-normalized heatmap with cell counts and row percentages.	Confusion counts diagnose this run only and do not imply full-dataset generalization.
fig. 17	class diagnostics	Ranked off-diagonal confusion-pair bars with true-class error rates.	Pair counts identify local error cases and are not a general taxonomy.
fig. 18	class diagnostics	Grouped train/test accuracy and loss bars by evaluated candidate.	Gaps are local bounded-loop diagnostics, not convergence guarantees.
fig. 10	training history	Epoch-level held-out accuracy lines with accepted best-epoch marker.	Learning curves diagnose configured training only, not convergence in general.
fig. 22	paired comparison	Matched accepted-versus-baseline correctness heatmap.	Matched comparison is limited to the fixed local test split.
fig. 9	confusion matrix	Per-class accuracy bars computed from the confusion matrix diagonal.	Per-class values diagnose this local split only and do not certify robustness.
fig. 20	probability diagnostics	Confidence and margin histograms split by correctness.	Distributions are descriptive diagnostics for the fixed local test split.
fig. 24	statistical summary	Brier score and negative-log-likelihood bar comparison.	Probability-quality metrics compare the configured evaluated candidates only.
fig. 19	robustness report	Candidate-by-transform accuracy heatmap for deterministic perturbations.	Deterministic perturbations are a smoke test and do not certify robustness.
fig. 23	statistical summary	Confidence-threshold coverage and selective-accuracy line chart.	Selective accuracy describes thresholded predictions on the fixed local split only.

Evidence And Scoring III

fig. 11	training diagnostics	Final and best-epoch accuracy bars plus train-test gap bars.	Training dynamics diagnose this configured deterministic run only.
fig. 1	class balance	Grouped train/test class-count bars from the local MNIST fixture.	Class counts describe the local fixed fixture and are not population statistics.
fig. 13	error examples	Deterministic grid of the first accepted-candidate misclassifications.	Examples are qualitative diagnostics for this run, not an error taxonomy.
fig. 2	small	Class-balanced contact sheet from fixed local MNIST arrays.	The sheet illustrates the local fixed subset and is not a statistical sample claim.